CORTO 3

Integrantes:

María José Yee Vidal

Sebastian Estrada Tuch

OBJETIVO

Implementar una versión simplificada del modelo productor–consumidor para identificar condiciones de carrera y comparar el uso de atomic y critical en la protección de variables compartidas.

INSTRUCCIONES

El modelo productor–consumidor representa una situación en la que dos o más procesos o hilos trabajan con un recurso compartido. Un productor se encarga de generar datos o elementos, mientras que un consumidor utiliza o procesa los elementos generados. Ambos trabajan de manera concurrente, por lo que es necesario coordinar sus acciones:

El productor (generará productos representados mediante números) no debería reemplazar un producto antes de que este haya sido consumido. El consumidor (leerá y consumirá cada producto) no debería intentar utilizar un producto que todavía no ha sido generado. Ambos accederán a una misma variable compartida. Las barreras permitirán coordinar el orden de producción y consumo.

---

1. Antes de ejecutar el programa, analicen el código y respondan: ProducerConsumer.c

¿Qué hilo desempeña el papel de productor y cuál el papel de consumidor?

El hilo identificado con 0 es el productor y el hilo identificado con 1 es el consumidor.

¿Cuál es la variable compartida entre ambos hilos?

La variable compartida entre hilos es el de 'producto' en 'shared(producto)'.

¿Qué instrucción asigna un nuevo valor al producto?

'producto = i', esto asigna al producto los valores entre 1 y 5.

¿Qué instrucción representa el consumo del producto?

'printf("Consumidor: consumio el producto %d\n", producto);'

Representa el consumo pq el consumidor lee y muestra el valor almacenado en 'producto'.

¿Cuál es la función de la primera barrera y la segunda?

La primera barrera tiene la función de obligar al consumidor a esperar hasta que el productor haya generado el producto de la iteración actual.

Y la segunda barrera tiene la función de obligar al productor a esperar hasta que

el consumidor ya haya consumido el producto actual antes de generar el siguiente.

---

2. Ejecución y observación del programa

Compilen el programa productor–consumidor proporcionado, y ejecútenlo tres veces y observen el orden de los mensajes. Completen la siguiente tabla:

Número de Ejecución	¿Se produjeron los productos del 1 al 5?	Cada producto fue consumido después de producirse?	¿El orden fue correcto?
1	Sí	Sí	Sí
2	Sí	Sí	Sí
3	Sí	Sí	Sí

Expliquen brevemente cómo se alternan el productor y el consumidor durante cada iteración.

Durante cada iteración, el hilo productor asigna un valor a 'producto' y muestra el mensaje de producción, luego, la primera barrera permite al consumidor leer el producto, la segunda barrera impide que el productor genere producto antes de que el actual haya sido consumido. De ahí viene la alternación en el orden producir-consumir.

---

3. Comenten o eliminen temporalmente las dos barreras, compilen nuevamente y ejecuten el programa tres veces.

En producerconsumer.c:

```c
// #pragma omp barrier
```

Número de Ejecución	¿Se produjeron los productos del 1 al 5?	Cada producto fue consumido después de producirse?	¿El orden fue correcto?
1	Sí	Sí	Sí
2	Sí	Sí	Sí
3	Sí	Sí	Sí

Respondan:

¿Qué diferencias encontraron respecto de la versión con barreras?

En las tres ejecuciones realizadas no se observaron diferencias visibles. Los productos del 1 al 5 fueron producidos y consumidos en el orden correcto. Aunque, al eliminar las barreras este orden deja de estar garantizado.

¿Por qué eliminar las barreras afecta el resultado?

Eliminar las barreras elimina la sincronización entre el productor y el consumidor, entonces los dos hilos pueden acceder concurrentemente a 'producto', por lo que el productor podría reemplazar su valor antes de que sea consumido y el consumidor puede leerlo antes de que se produzca.

---

4. Protección con atomic: ProducerConsumerAtomic.c

La directiva atomic permite proteger una operación sencilla de lectura o escritura sobre una variable compartida. De esta forma, un hilo no puede leer la variable producto mientras otro hilo está escribiendo en ella.

Respondan las siguientes preguntas:

¿Qué operación protege atomic write?

Protege la escritura: `producto = i;`

¿Qué operación protege atomic read?

Protege la lectura: `productoConsumido = producto;`

¿El productor y el consumidor volvieron a alternarse?

No, porque en la ejecución obtenida el consumidor leyó 1, 4, 4, 5 y 5, omitiendo el producto 2 y 3, y repitiendo otros. Aunque atomic protege las lecturas y escrituras, no obliga al productor a esperar al consumidor, no garantizando la alternancia.

---

5. Coordinación utilizando critical

Se usa critical para proteger como una sola unidad la verificación, lectura o escritura del producto y el cambio de su estado: ProducerConsumerCritical.c

Se agregó la variable compartida hayProducto:

- 0 representa que no hay un producto disponible.
- 1 existe un producto esperado a ser consumido.

Compilen y ejecuten el programa tres veces.

Número de Ejecución	¿Se produjeron los productos del 1 al 5?	Cada producto fue consumido después de producirse?	¿El orden fue correcto?
1	Sí	Sí	Sí
2	Sí	Sí	Sí
3	Sí	Sí	Sí

Respondan las siguientes preguntas:

¿Cuál es la función de la variable hayProducto?

Su función es indicar el estado del recurso compartido, el valor 0 significa que no hay un producto disponible y el valor 1 significa que existe un producto esperando por ser consumido.

¿Por qué el productor solamente produce cuando hayProducto es igual a 0?

Porque 0 indica que el espacio compartido está disponible, evitando que el productor reemplace un producto que todavía no haya sido consumido.

¿Por qué el consumidor solamente consume cuando hayProducto es igual a 1?

Porque 1 indica que ya existe un producto disponible, similar a lo de antes, evita que el consumidor lea un producto que todavía no haya sido generado o ya consumido.

¿Qué instrucciones se encuentran protegidas por critical?

En el productor protege la verificación de 'hayProducto', escritura de 'producto', cambio de 'hayProducto', cambio de 'producto' y el print del mensaje.

En el consumidor protege la verificación de 'hayProducto', lectura e impresión de 'producto', cambio de 'hayProducto' a 0 y el cambio a consumido.

Salida de ejecución

```
$ ./ProducerConsumer.exe
Productor: produjo el producto 1
Consumidor: consumio el producto 1
Productor: produjo el producto 2
Consumidor: consumio el producto 2
Productor: produjo el producto 3
Consumidor: consumio el producto 3
Productor: produjo el producto 4
Consumidor: consumio el producto 4
Productor: produjo el producto 5
Consumidor: consumio el producto 5

mjyee@MaJo_ UCRT64 /c/Users/mjyee/Downloads/CC3069-Computacion-Paralela/corto3
$ gcc ProducerConsumerAtomic.c -o ProducerConsumerAtomic.exe -fopenmp

mjyee@MaJo_ UCRT64 /c/Users/mjyee/Downloads/CC3069-Computacion-Paralela/corto3
$ ./ProducerConsumerAtomic.exe
Productor: produjo el producto 1
Productor: produjo el producto 2
Productor: produjo el producto 3
Consumidor: consumio el producto 1
Consumidor: consumio el producto 4
Productor: produjo el producto 4
Productor: produjo el producto 5
Consumidor: consumio el producto 4
Consumidor: consumio el producto 5
Consumidor: consumio el producto 5

mjyee@MaJo_ UCRT64 /c/Users/mjyee/Downloads/CC3069-Computacion-Paralela/corto3
$ gcc ProducerConsumerCritical.c -o ProducerConsumerCritical.exe -fopenmp

mjyee@MaJo_ UCRT64 /c/Users/mjyee/Downloads/CC3069-Computacion-Paralela/corto3
$ ./ProducerConsumerCritical.exe
Productor: produjo el producto 1
Consumidor: consumio el producto 1
Productor: produjo el producto 2
Consumidor: consumio el producto 2
Productor: produjo el producto 3
Consumidor: consumio el producto 3
Productor: produjo el producto 4
Consumidor: consumio el producto 4
Productor: produjo el producto 5
```

```
Consumidor: consumio el producto 5

$ gcc ProducerConsumer.c -o ProducerConsumerSinBarreras.exe -fopenmp

mjyee@MaJo_ UCRT64 /c/Users/mjyee/Downloads/CC3069-Computacion-Paralela/corto3
$ ./ProducerConsumerSinBarreras.exe
Productor: produjo el producto 1
Consumidor: consumio el producto 1
Productor: produjo el producto 2
Consumidor: consumio el producto 2
Productor: produjo el producto 3
Consumidor: consumio el producto 3
Productor: produjo el producto 4
Consumidor: consumio el producto 4
Productor: produjo el producto 5
Consumidor: consumio el producto 5

mjyee@MaJo_ UCRT64 /c/Users/mjyee/Downloads/CC3069-Computacion-Paralela/corto3
$ ./ProducerConsumerSinBarreras.exe
Productor: produjo el producto 1
Consumidor: consumio el producto 1
Productor: produjo el producto 2
Consumidor: consumio el producto 2
Productor: produjo el producto 3
Consumidor: consumio el producto 3
Productor: produjo el producto 4
Consumidor: consumio el producto 4
Productor: produjo el producto 5
Consumidor: consumio el producto 5

mjyee@MaJo_ UCRT64 /c/Users/mjyee/Downloads/CC3069-Computacion-Paralela/corto3
$ ./ProducerConsumerSinBarreras.exe
Productor: produjo el producto 1
Consumidor: consumio el producto 1
Productor: produjo el producto 2
Consumidor: consumio el producto 2
Productor: produjo el producto 3
Consumidor: consumio el producto 3
Productor: produjo el producto 4
Consumidor: consumio el producto 4
Productor: produjo el producto 5
Consumidor: consumio el producto 5
```